task1

August 23, 2026

[1]:
```
import numpy as np
import pandas as pd

df = pd.read_csv("udemy_courses_with_student_ids_and_ratings.csv")
course_map = df[['course_id', 'course_title']].drop_duplicates().
 ↪set_index('course_id')['course_title'].to_dict()
print(df.head())
```

```
    student_id  course_id                                       course_title  \
0  STU01FBPRI1    1070968                 Ultimate Investment Banking Course
1  STUF3HFG09C    1113822  Complete GST Course & Certification - Grow You…
2  STUJG8GTUDT    1006314  Financial Modeling for Business Analysts and C…
3  STU92SONB81    1210588  Beginner to Pro - Financial Analysis in Excel …
4  STURBOOMLKP    1011058       How To Maximize Your Profits Trading Options

                                                 url  is_paid  price  \
0  https://www.udemy.com/ultimate-investment-bank…     True    200
1     https://www.udemy.com/goods-and-services-tax/     True     75
2  https://www.udemy.com/financial-modeling-for-b…     True     45
3  https://www.udemy.com/complete-excel-finance-c…     True     95
4  https://www.udemy.com/how-to-maximize-your-pro…     True    200

   num_subscribers  num_reviews  num_lectures             level  \
0             2147           23            51          All Levels
1             2792          923           274          All Levels
2             2174           74            51  Intermediate Level
3             2451           11            36          All Levels
4             1276           45            26  Intermediate Level

   content_duration   published_timestamp           subject  course_rating
0               1.5  2017-01-18T20:58:58Z  Business Finance            4.3
1              39.0  2017-03-09T16:34:20Z  Business Finance            4.6
2               2.5  2016-12-19T19:26:30Z  Business Finance            4.4
3               3.0  2017-05-30T20:07:24Z  Business Finance            1.5
4               2.0  2016-12-13T14:57:18Z  Business Finance            1.2
```

[2]:
```
A_df = df.pivot(index="student_id", columns="course_id", values="course_rating")
```

```
# Mean of ratings for each student
user_means = A_df.mean(axis=1)

# Subtract mean form their active rating to center the data and replace NaN␣
 ↪with zero
A_centered_df = A_df.sub(user_means, axis=0).fillna(0)

print(A_centered_df.head())
```

```
course_id     8324     8325     11153    11174    11475    12214    12975    \
student_id
STU00CN0KCP      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU00MJMZ1J      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU013OQ715      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU01D55K29      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU01FBPRI1      0.0      0.0      0.0      0.0      0.0      0.0      0.0

course_id     13216    14571    15285    …  1273896  1274210  1274846  \
student_id                                …
STU00CN0KCP      0.0      0.0      0.0   …      0.0      0.0      0.0
STU00MJMZ1J      0.0      0.0      0.0   …      0.0      0.0      0.0
STU013OQ715      0.0      0.0      0.0   …      0.0      0.0      0.0
STU01D55K29      0.0      0.0      0.0   …      0.0      0.0      0.0
STU01FBPRI1      0.0      0.0      0.0   …      0.0      0.0      0.0

course_id     1275790  1275872  1276020  1276182  1276364  1277924  1282064
student_id
STU00CN0KCP      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU00MJMZ1J      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU013OQ715      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU01D55K29      0.0      0.0      0.0      0.0      0.0      0.0      0.0
STU01FBPRI1      0.0      0.0      0.0      0.0      0.0      0.0      0.0

[5 rows x 3672 columns]
```

[3]:
```
# Convert to numpy array
A = A_centered_df.values

AtA = np.dot(A.T, A)

eigenvalues, eigenvectors = np.linalg.eigh(AtA)

sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]

# Extract to k = 2 components
```

```python
k = 2
top_eigenvalues = eigenvalues[:k]
V_k = eigenvectors[:, :k]


sigma_k = np.sqrt(np.maximum(top_eigenvalues, 0))
sigma_mat = np.diag(sigma_k) # Diagonal matrix Sigma i

# U_i = (A V_i) / sigma_i
U_k = np.zeros((A.shape[0], k))

for i in range(k):
    # To prevent division by zero for sparse matrices
    if sigma_k[i] > 1e-9:
        U_k[:, i] = np.dot(A, V_k[:, i]) / sigma_k[i]
    else:
        U_k[:, i] = 0

# 8. Dense Reconstruction: A_hat = U_k * Sigma_k * V_k^T + User Means
# Note: V_k is transposed for the reconstruction formula requires V^T
A_reconstructed = np.dot(np.dot(U_k, sigma_mat), V_k.T) + user_means.values.
 ↪reshape(-1, 1)

# Convert back to a DataFrame for Task 3
A_pred_df = pd.DataFrame(A_reconstructed, index=A_centered_df.index,␣
 ↪columns=A_centered_df.columns)

print("Reconstructed Dense Matrix (First 5 Rows):")
print(np.round(A_pred_df.head(), 2))
```

```
Reconstructed Dense Matrix (First 5 Rows):
course_id    8324     8325     11153    11174    11475    12214    12975    \
student_id
STU00CN0KCP      4.5      4.5      4.5      4.5      4.5      4.5      4.5
STU00MJMZ1J      2.1      2.1      2.1      2.1      2.1      2.1      2.1
STU013OQ715      3.0      3.0      3.0      3.0      3.0      3.0      3.0
STU01D55K29      4.5      4.5      4.5      4.5      4.5      4.5      4.5
STU01FBPRI1      4.3      4.3      4.3      4.3      4.3      4.3      4.3

course_id    13216    14571    15285    …  1273896  1274210  1274846  \
student_id                             …
STU00CN0KCP      4.5      4.5      4.5  …      4.5      4.5      4.5
STU00MJMZ1J      2.1      2.1      2.1  …      2.1      2.1      2.1
STU013OQ715      3.0      3.0      3.0  …      3.0      3.0      3.0
STU01D55K29      4.5      4.5      4.5  …      4.5      4.5      4.5
STU01FBPRI1      4.3      4.3      4.3  …      4.3      4.3      4.3
```

```
course_id    1275790  1275872  1276020  1276182  1276364  1277924  1282064
student_id
STU00CN0KCP      4.5      4.5      4.5      4.5      4.5      4.5      4.5
STU00MJMZ1J      2.1      2.1      2.1      2.1      2.1      2.1      2.1
STU013OQ715      3.0      3.0      3.0      3.0      3.0      3.0      3.0
STU01D55K29      4.5      4.5      4.5      4.5      4.5      4.5      4.5
STU01FBPRI1      4.3      4.3      4.3      4.3      4.3      4.3      4.3

[5 rows x 3672 columns]
```

[4]:
```python
def recommend_top_courses(student_id, top_n=3):
    if student_id not in A_pred_df.index:
        return "Student ID not found in the training data"

    user_predictions = A_pred_df.loc[student_id]
    interacted_courses = A_df.loc[student_id].dropna().index

    unseen_predictions = user_predictions.drop(interacted_courses)

    top_course_ids  = unseen_predictions.sort_values(ascending=False).
 ↪head(top_n)

    recommendations = [
        (course_map[c_id], round(score, 2))
        for c_id, score in top_course_ids.items()
    ]

    return recommendations

target_student = "STU01FBPRI1"
print(f"Top {3} recommendations fro {target_student}:")
results = recommend_top_courses(target_student, top_n=3)

if isinstance(results, str):
    print(results)
else:
    for rank, (title, predicted_rating) in enumerate(results, 1):
        print(f"{rank}. {title} (Predicted Score: {predicted_rating})")
```

```
Top 3 recommendations fro STU01FBPRI1:
1. Javascript for Beginners (Predicted Score: 4.3)
2. HTML Tutorial: HTML & CSS for Beginners (Predicted Score: 4.3)
3. Learn jQuery: An In-depth Course For Beginners (Predicted Score: 4.3)
```